Scheduler in C with Shell integrated

Group Members

1. **Nihal, 2023345**
2. **Namit Bajaj, 2023340**

Contribution Summary

- Equal Contribution from Both Members weather in implementation, debugging etc.

Scheduler Overview

The **scheduler** is responsible for managing the execution of multiple processes based on a round-robin scheduling algorithm. Here are the key components and functionality:

1. **Shared Memory**:
    - The Code utilizes shared memory to communicate between the scheduler and the shell and process and the executed processes.
2. **Process Information Storage**:
    - Each process's details, including its name, PID, completion time, and wait time, are stored in a structured format within shared memory. This enables the scheduler to maintain state information about each process.
3. **Signal Handling**:
    - The scheduler implements signal handling to manage process states effectively for RR scheduling. It responds to signals that control process execution, allowing it to pause and resume processes as needed.
4. **Round-Robin Scheduling**:

- The scheduler operates using a round-robin algorithm, allocating a fixed time slice to each process. If a process does not complete within its allocated time, it is paused and placed at the back of the queue.

5. **Process Execution**:
   - When a command is submitted using the shell, the code forks a new process for the command execution and stores its details in shared memory. Another Fork manages the output of these commands, allowing users to see the results in real-time.
6. **Termination Handling**:
   - The code handles the termination of processes. When a process finishes execution, it is removed from the queue, and the scheduler updates the relevant metrics (e.g., wait times and completion times).

Shell Program Overview

The shell program operates within the same file as the scheduler and is designed to accept submission commands only. Here are its key features:

1. **Command Submission**:
   - The shell will be initialzed with shell executable

     Gcc -o shell shell.c -lm

     And ./shell <ncpu> <time slice>

   - The shell prompts the user to submit jobs, which are then passed to the scheduler for execution. It accepts commands in the format submit <jobs>.

     Eg: submit ./a

     a is an executable

2. **Input Handling**:
   - The shell reads user input and processes the command. If the CTRL-C is given, the shell will terminate printing the process summary from scheduler.

3. **Feedback Mechanism**:
    - Upon submission of a command, the shell provides feedback indicating that the process is being handled

---

Dummy Executables

1. **Factorial Executable**:
    - This program calculates the factorial of 20.

Fibonacci Executable:

- This program computes Fibonacci of 40

They are implemented with dummy_main.h

The dummy_main.h file is essential for:

1. **Unified Entry Point**: Provides a consistent starting point for all executables.
2. **Scheduler Integration**: Ensures smooth communication with the scheduler.
3. **Simplified Management**: Standardizes process tracking.
4. **Modularity**: Separates computation and scheduling logic.
5. **Easier Testing**: Facilitates validation within the scheduler.
6. **Custom Initialization**: Allows for specific setup for each executable.
